

新卒1年目の教科書：社会人としての土台を築く教訓

このドキュメントは、現場の先輩たちの実体験に基づく「振り返りシート」を体系化したものです。「先輩たちはなぜそのルールにこだわるのか？」という理由を知ること、迷わず行動できるようになります。現場で困ったときの道標として活用してください。

作成日：2026/01/28

参考文献：[新卒一年目_振り返りシート](#)

第1章：【思考と伝達】プロフェッショナルの基本作法

忙しい上司や先輩は、常に多くの判断を迫られています。あなたの報告が分かりにくいと、相手は理解するだけで疲れてしまいます。

読みやすい文章を作成することを心がけてください。

1. 一文を短くする

「Aなのですが、Bで、CなのでDです」と繋げず、短く切ります。

- **良い例**：「AはBです。そのためCとなります。結果としてDです」
- **理由**：一文が長いと、主語と述語の関係が分かりにくくなり、読み間違いの原因になるからです。

2. 読点（、）を意識的に打つ

息継ぎをする場所や、意味の切れ目に必ず「、」を入れます。

- **理由**：ひらがなや漢字が続くと読みづらく、視覚的なストレスを与えるからです。

3. 結論から話す（結論ファースト）

「経緯」から話さず、「結果」や「相談したいこと」から話します。

- **理由**：聞き手は「何の話をされているのか（報告？相談？トラブル？）」が分からないまま話を聞くのが一番ストレスだからです。

4. 箇条書きを活用する

ダラダラと文章で書かず、リスト形式で見やすくします。

- **理由**：情報の構造が一目で分かり、重要なポイントが埋もれないようにするためです。

5. 定義された用語とURLを使う

画面名を呼ぶときは自分勝手な呼び方をせず、仕様書（Figmaなど）にある正式名称とURLをセットで伝えます。

- **理由**：「あの画面」と言ったとき、あなたと他の人で違う画面を想像していると、話が噛み合わず作業のやり直しが発生するからです。

6. 「場合」と「とき」を使い分ける

- **場合**：前提条件や、全体のケースを指すときに使います。（例：エラーが発生した場合）
- **とき**：具体的な瞬間やアクションを指すときに使います。（例：ボタンを押したとき）
- **理由**：言葉の定義を曖昧にすると、仕様書や報告の論理が破綻するからです。

7. 「これ・あれ」を使わない（固有名詞の徹底）

代名詞（こそあど言葉）を避け、具体的な名前呼びます。

- **理由**：指しているものがズれていることに気づかないまま作業が進むと、後で大きなトラブルになるからです。

8. 送信前に「声出し確認」をする

チャットやメールを送る前に、10秒でいいので書いた文章を声に出して読みます。

- **理由**：目で見ても気づかない誤字や、日本語の不自然さは、耳で聞くと気づきやすいからです。

9. 違和感を無視しない

指示を聞いて「あれ？何かおかしいな」と思ったら、その場ですぐに確認します。

- **理由**：「なんか変だな」という直感は、意外と当たっています。それを放置して進めると、後で取り返しのつかないミスになります。

第2章：【タスクの進め方】 成果を最大化する実行力

仕事において期待値のコントロールは重要です。設定された期限を守れないなら、早めに報連相することを心がけてください。

1. 時間に余裕を持つ（バッファの確保）

作業時間の見積もりには、必ず期間的な余裕（バッファ）を持たせます。

- **理由：** 仕事中には急な会議や割り込みが必ず入ります。ギリギリの計画は必ず破綻するからです。

2. まず作業を少しだけやってみる

着手する前に全体像を把握するため、作業を少し（10%程度）やってみます。

- **理由：** 実際に取り組んでみて想定のタスク量と異なることに気がつくと、期限に間に合わなくなるからです。

3. 「15分考えて分からなければ聞く」ルール

自分で調べるのは最大15～30分まで(ケースによる)。それ以上悩んだら先輩に聞きましょう。

- **理由：** 新人が1時間悩んで解決しないことに関して、先輩なら1分で分かることがあるからです。悩み続けるのは時間の浪費です。

4. 資料作成の方向性を先に合わせる

成果物を作り込む前に、ある程度方向性が決まった段階で「こういう構成で作ろうと思いますが合っていますか？」と上司に確認します。

- **理由：** 全部作り終わってから「求めていたのと違う」と言われると、全ての時間が無駄になるからです。

5. 資料はレビューの時間を考慮し事前に送る

確認（レビュー）してほしい期限のある資料は、会議の直前ではなく、余裕を持って事前に送ります。

- **理由：** レビューも忙しいため、事前に目を通す時間を確保してもらうためです。

6. 1つ指摘されたら全体を見直す

「ここが間違っている」と言われたら、その箇所を直すだけでなく、他のページでも同じミスをしていないか全体を検索して直します。

- **理由：** 1箇所間違えているということは、自分の理解が間違っており、他の箇所でも同じミスをしている可能性が高いからです。

第3章：【会議と議事録】 チームを前に進める技術

会議は「何かを決めて、次に進むため」に行います。新人のうちは「議事録（会議の記録）」を担当することが多いですが、これは会議の構造を理解したり、文章を作成するトレーニングです。

1. 会議のゴールを最初に確認する

「今日、何が決まればこの会議は成功か」を確認します。

- **理由：** ゴールがないと話が脱線しやすく、時間を浪費してしまうからです。

2. 自分の仮説をぶつける

「どうすればいいですか？」といったオープンクエスションではなく、できる限りYes/Noで答えられるクローズドクエスションで聞きます。

- **良い例：** 「私は〇〇だと思うのですが、認識合っておりますか？」
- **理由：** 相手は「Yes/No」で答えるだけで済むので、負担が減り、会話がスムーズになるからです。

3. 議事録は「決定事項」と「ToDo」を明確にする

誰が・いつまでに・何をやるか（ToDo）を必ず書きます。

- **理由：** これを書かないと、「いい話をしたけれど、結局誰も動かない」という状態になるからです。

4. 議事録にメリハリをつける（MECE感）

全ての会話を同じ量で記録するのではなく、重要な決定事項は詳しく、軽い報告は1行で済ませるなど強弱をつけます。

- ※MECE（ミーシー）：漏れなく、ダブリなくという意味。議題A、B、Cを全て同じ行数・詳しさと書く（読みにくい）。
- **良い例：**
 - **議題A（重要・決定事項あり）**：背景・決定プロセス・ToDoまで詳細に記述。
 - **議題B（報告のみ）**：「共有済み、異議なし」と一行で要約。
 - **議題C（次回持ち越し）**：「次回検討」とし、詳細は割愛。
- **理由：** 全部を詳細に書くと長すぎて誰も読まないからです。「重要なこと」がパッと分かるのが良い議事録です。

5. Googleカレンダー設定時のマナー（権限設定）

- **Googleカレンダー設定の鉄則：** 他人の時間を奪わないよう、以下の手順で設定します。詳しくは[DocBase](#)を参照してください。
 1. **「時間を探す」タブの活用：** ゲストを追加した後、全員の空き状況を確認します。被っている時間に無理やり入れないようにしましょう。
 2. **タイトルの命名規則：** **【meet】プロジェクト名_議題** のように、招待された側が一目で内容を理解できる件名にします。
 3. **ゲスト権限の委譲（重要）：** Google Meet/カレンダー作成画面の右下にある「ゲストの権限」設定で、「予定を変更する」に**チェック**を入れます。これにより、急な変更が必要になった際、主催者以外でも時間の調整が可能になり、再調整の手間が減少します。

4. **場所と詳細：** リモートなら「Google Meetに参加」ボタンを忘れずに生成します。「説明」欄には、アジェンダや参照資料のURLを貼り付けてください。

第4章：【開発・エンジニアリング】品質と信頼を担保する規律

「誰のPCでも動く」「いつでも元に戻せる」状態を保つことを心がけましょう。

1. 開発環境を汚さない

プロジェクトごとに必要なツールだけを入れる設定（direnvなど）を使います。

- **理由：** 自分のPC全体（グローバル）にツールを入れると、別のプロジェクトで「バージョンが合わない」などの不具合が起きるからです。

2. 便利なツールでミスを防ぐ

VSCoode（プログラムを書くソフト）の拡張機能を使い、スペルミスや不要なスペースなど、手間を省く機能を活用してください。

- **理由：** 人間は必ずミスをします。機械に任せられるチェックは機械にやらせて、人間はもっと重要な思考に集中するためです。
- **参考：** [Visual Studio Codeに入れるべき拡張機能](#)

3. ターミナルにブランチ名を表示する

今どの「ブランチ（作業場所）」にいるかを常に画面に表示させます。

- **理由：** 間違った場所（本番環境など）でコマンドを実行してデータを消してしまう事故を防ぐためです。
- **参考：** [ターミナルに現在作業中のGitブランチ名を表示](#)

4. PR（プルリクエスト）を見やすくする

自分のコードを確認してもらう依頼（PR）には、タスクのURLや、確認してほしいポイントを整理して書きます。プロジェクトごとにルールが定められているため、必ず確認してください。

- **理由：** 確認する人（レビュワー）が「何のための変更か」を理解する時間を減らし、早く承認をもらうためです。

5. 「元に戻す手順」を用意する

データベースなど、何かデータやコードを変更する際は、必ず「失敗したときに元に戻すための命令や方法」を用意してから作業します。

- **理由：** 作業中にエラーが出た時、すぐに元の状態に戻せないと、サービス停止や手戻りなどに繋がるからです。

6. テスト仕様書を見やすくする

テストの手順書は、1つの行に1つの動作だけを書きます。

- **理由：** 「AをしてBを確認する」とまとめて書くと、テストをする人が「AはしたけどBの確認を忘れた」というミスをしやすくなるからです。